ARCHITECTURE & DESIGN · PART 1

Collaborate Authorization Layer

A federation broker, a permission engine, and an on-behalf-of token exchange — built around Caseware's identity provider rather than replacing it.

ROLE	COVERS	STACK	OUT OF SCOPE
Senior Developer, Collaborate	Login · Permissions · Delegation	ASP.NET Core · AWS · Redis	Credential storage, MFA

ASSUMPTIONS

- Caseware's central IdP is an external OIDC dependency (discovery / token / userinfo). Not built here.
- Workspace roles, resource-level overrides, and firm policy live in Collaborate's database, which can emit change events.
- Downstream services validate tokens only. They never read the permissions database.
- Redis (ElastiCache) and AWS infrastructure are available.

01 High-Level Architecture

Three components, each owning one of the three problems. They share one substrate: Collaborate's own authorization server is the only token issuer that downstream services trust, and the permissions database remains the only source of truth for what a user may do.

A Federation Broker

Collaborate runs its own OAuth2/OIDC authorization server. It federates upstream per firm — firm staff to Caseware's central IdP, external users to their own firm's SAML or OIDC provider. Authorization Code + PKCE runs between the client and *our* AS; the upstream handshake is a separate, internal concern.

Per-firm configuration — issuer, client credentials or SAML metadata, claim mapping, redirect URIs — lives in Collaborate's database, cached in front. Firm is resolved *before* redirect from the workspace or invite context already present in the URL, with email-domain home-realm discovery as the fallback for generic entry points.

> **ASP.NET CORE · DYNAMIC SCHEMES**
>
> Authentication schemes are registered at firm onboarding through `IAuthenticationSchemeProvider`. Routine config changes — secret rotation, cert renewal — do **not** re-register: a change event evicts the entry from `IOptionsMonitorCache<T>` and the next request rebuilds it from current values. Registration is a firm-lifecycle operation; options refresh is a config-change operation. Conflating the two produces needless churn on every minor edit.

B Policy Decision Point

Access tokens carry **identity and coarse scope only**. Fine-grained state — workspace role, resource-level overrides, firm policy — is resolved per request by the PDP.

> If a fact can change between the moment a token is minted and the moment it is used, it does not belong in the token.

Embedding permissions would make token lifetime the revocation SLA, and per-document overrides cannot be enumerated into a header anyway. So the PDP caches **inputs, not decisions**: one Redis read returns `{ role, resourceOverrides[], firmPolicyVersion }`, and a pure in-process function maps that plus the requested resource and action to permit or deny.

Caching resolved decisions instead — `can:{user}:{resource}:{action}` — would explode cardinality, turn a single role change into thousands of key deletions, and move authorization logic into cache state where it cannot be unit-tested.

Invalidation is by delete, never update. The next read repopulates from the source of truth, so a bug in an update path can never leave the cache silently disagreeing with the database. Firm policy changes would otherwise touch every user in a firm, so cached entries carry a `firmPolicyVersion` stamp and a mismatch is treated as a miss — mass invalidation becomes a counter bump rather than a key scan.

Enforcement sits at Collaborate's API layer, which consults the PDP and then calls downstream services with an audience-pinned token. Those services accept *only* tokens minted by our AS for their own audience, so a client cannot bypass the enforcement point by calling the Document Service directly. The PDP is additionally exposed over HTTP for services that genuinely need to ask.

C On-Behalf-Of · RFC 8693

One endpoint and one rule set serves both delegation scenarios — a client's system acting for its employee, and an internal service acting for the user who triggered it. The caller presents its own client credentials plus a `subject_token`; the AS returns a JWT with `sub` set to the user, `act` to the calling party, `aud` pinned to one downstream service, and narrowed scope.

FIVE CONTROLS AGAINST CONFUSED DEPUTY

Audience pinning	`aud` is exactly one service. Without it, a token minted for the notification service replays against the Financial Data API.
Scope as intersection	`requested ∩ subject token ∩ client ceiling ∩ current PDP`. A bug here is privilege escalation, which is why the logic is kept pure and exhaustively tested.
Actor entitlement	May this client act for this user at all? The commonly omitted control — and precisely where the vulnerability lives.
Short lifetime	60–300 seconds, so exchanged tokens expire faster than the revocation SLA and need no revocation infrastructure of their own.
Delegation depth cap	One hop, counted from nested `act` claims. Without it a delegated token can be re-delegated onward toward a higher-trust audience.

TWO DELIBERATE DEVIATIONS FROM RFC 8693

The spec makes `audience` optional (§2.1); we **require** it, because an unpinned token is the confused-deputy problem restated — no audience, no token.

And §4.4's `may_act` claim, the spec's own mechanism for recording who may act for a subject, is **not used**: an embedded claim freezes the decision at issue time, which is exactly the staleness this design avoids everywhere else. The entitlement lookup runs at exchange time instead, so a withdrawn authorization takes effect immediately.

One rule falls out of the same reasoning: when no scope survives the intersection the exchange refuses, rather than issuing a scopeless token that a downstream service might read as unrestricted.

The sharp edge is the external-client scenario: **a client must be structurally incapable of acting for a user outside its own firm.** The client record already carries `firmId` from the broker design, so the exchange refuses when `subject.firmId ≠ client.firmId`. This is a structural invariant with a dedicated test, not a policy row someone can misconfigure. First-party internal services that legitimately act across firms are modelled as an explicit, separately-named actor constraint rather than a quiet exception to the same check.

The exchange consults the PDP at mint time, so scope reflects what the user can do *now*. That, plus the short lifetime, is how "fast" and "consistent with the source of truth" are reconciled — and it keeps the exchange a PDP consumer rather than a second source of truth.

02 Implementation Plan

PHASE	DELIVERABLE
01	Broker AS with the central-IdP path only; per-firm client store; PKCE. Downstream services switched to trusting one issuer.
02	Per-firm federation: dynamic scheme registration, claim mapping, one pilot firm — OIDC first, then SAML.
03	PDP: pure evaluation function, Redis cache, DB change hooks, enforcement point in the API layer. Shadow mode — evaluate and log, enforce nothing.
04	Enforcement on, plus revocation event fanout to long-lived connections.
05	Token exchange endpoint; migrate internal services off token forwarding.

Phase 3's shadow mode is the load-bearing step: it lets the decision function be validated against real production traffic before any request can be denied by it.

OPEN DECISION · LIBRARY

Duende IdentityServer is more mature and carries commercial support, but is licensed per registered client application — the same axis this system grows along, since per-firm client configuration means firm onboarding drives client count. OpenIddict has no licensing cost at any scale, at the price of hand-building login and consent surfaces. Worth noting that RFC 8693 is a custom grant handler in *both* — neither ships it natively — so the exchange work is identical either way.

03 Testing Strategy

- **Pure functions, table-driven.** Scope intersection and the permission evaluator are pure by construction, so the full matrix — role × override × firm policy × action — is testable with no infrastructure. The security-critical logic is deliberately the part kept free of I/O.
- **Negative tests as first-class.** Cross-firm delegation, scope escalation, replay at the wrong audience, expired subject tokens — all refused. Plus the case that proves PDP consultation happens at mint time: a scope the user has *lost* is dropped even though the subject token still carries it.
- **Invalidation timing.** Assert a permission change reaches a denied decision inside the target window — measured, not assumed.
- **Load** against the PDP at target throughput with a realistic key distribution, including a deliberate hot-workspace case.

04 Evaluation & Observability

"Revocation takes effect within seconds" is an aspiration unless it is measured. The primary SLI is **propagation lag** — permission-change commit → cache invalidated → connected sessions re-checked — tracked as a distribution and alerted on p99.

- PDP decision latency p50/p99, and cache hit ratio.
- **Denies by reason.** A spike is either a misconfiguration or an attack; both warrant a page.
- **Cross-firm refusals at the exchange endpoint should sit at zero.** Any non-zero rate is a broken integration or an attempted confused-deputy attack, and is worth investigating individually rather than graphing.
- Database fallback rate and circuit-breaker state, as the early warning for cache degradation.
- **Immutable audit log of every exchange**, recording `sub` and `act` — attributability to the specific user is a stated requirement, not just a debugging aid.

05 Failure Modes & Tradeoffs

FAILURE MODE	RESPONSE
Redis unavailable	Fall back to the database behind a circuit breaker, with single-flight coalescing — without it, a cache outage converts directly into a database outage. Fail *closed* only when the database is also unreachable.
Invalidation event lost	Event delivery is the fast path, not the only path: cached entries also carry a modest TTL, so a dropped event self-heals within a bounded window instead of persisting indefinitely.
AS unavailable	A single point of failure by construction. Multi-AZ, stateless, horizontally scaled; degradation blocks new logins but does not invalidate live sessions.
A firm's IdP is down	Blast radius contained to that firm. The direct-federation alternative would have spread this across every downstream service.
Clock skew	Short-lived exchanged tokens are the most skew-sensitive component; require NTP and allow a small, explicit validation skew.

ACCEPTED TRADEOFFS

Cached authorization means a staleness window exists by definition; this design bounds and measures it rather than pretending otherwise. Enforcing at the API layer keeps downstream services simple but makes that layer security-critical. And the two scope-narrowing failure paths are handled differently on purpose — loud on escalation, silent on staleness — because collapsing them would either hide an attack signal or break legitimate integrations during ordinary permission churn.

AI usage on this exercise — where its output was corrected, and where it should not be trusted in this domain — is covered separately in the AI usage record.

Part 2 implements the on-behalf-of slice: an RFC 8693 token exchange endpoint in ASP.NET Core, leaning on the framework for token validation, signature verification, and signing — with custom code only for the exchange grant itself, which neither Duende nor OpenIddict provides natively.